

1. Objective

Implement K-Means clustering from scratch in Python — without using sklearn or pandas — and analyze how different values of K affect the clustering quality measured by inertia (SSE).

2. Dataset

A 2D synthetic dataset with 350 normalized data points, stored in dataset.txt. Each row contains two space-separated float values representing the x and y coordinates of a point in the range [0, 1].

- Total points: 350
- Features: 2 (x, y) — already normalized using min-max scaling
- No class labels — unsupervised learning task

3. Implementation Steps

- Load dataset from dataset.txt using numpy.
- Apply min-max normalization so all values fall between 0 and 1.
- Implement euclidean_distance() manually using a loop — no library functions.
- Randomly select K data points as initial cluster centers (seed = 11320065).
- Assign each point to its nearest center using Euclidean distance.
- Recalculate each center as the mean of all assigned points.
- Repeat assignment and update steps until centers shift less than 1e-6 or max 6 iterations.
- Compute inertia (SSE) as the sum of squared distances from each point to its cluster center.
- Run the algorithm for K = 2, 4, 6, 7 and record inertia for each.

4. Results — K vs. Inertia

The table below shows how inertia decreases as K increases:

K (Number of Clusters)	Inertia (SSE)	Change from Previous
K = 2	20.5739	—
K = 4	10.4624	↓ 10.11
K = 6	4.7415	↓ 5.72
K = 7	4.1669	↓ 0.57

5. Clustering Results for Each K

The figure below shows how the dataset is partitioned for each value of K. Black stars mark the final cluster centroids.

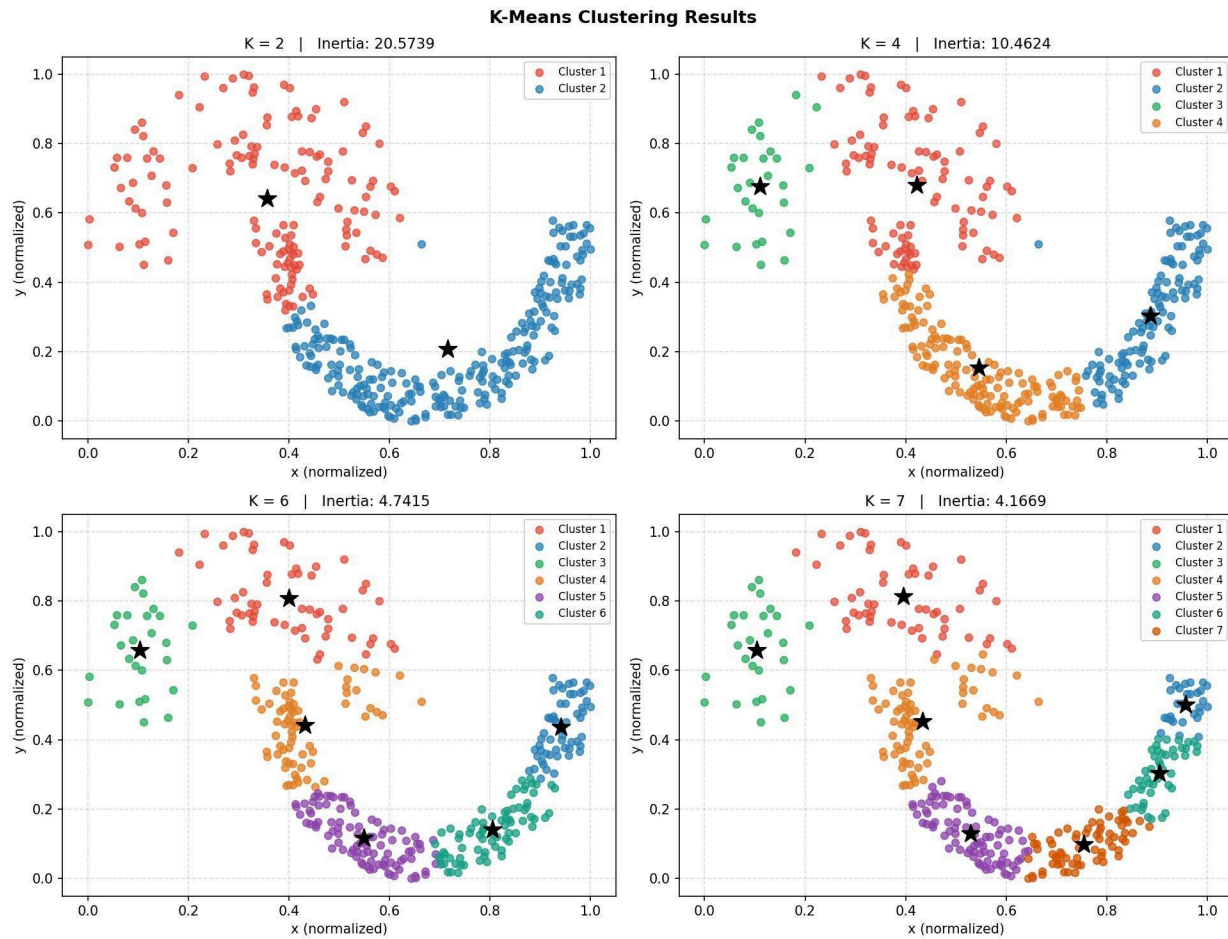


Figure 1: K-Means clustering results for K = 2, 4, 6, 7 with final centroids marked as stars

6. Inertia vs. K

The elbow curve shows how inertia drops sharply from K = 2 to K = 6, then flattens out after K = 6.

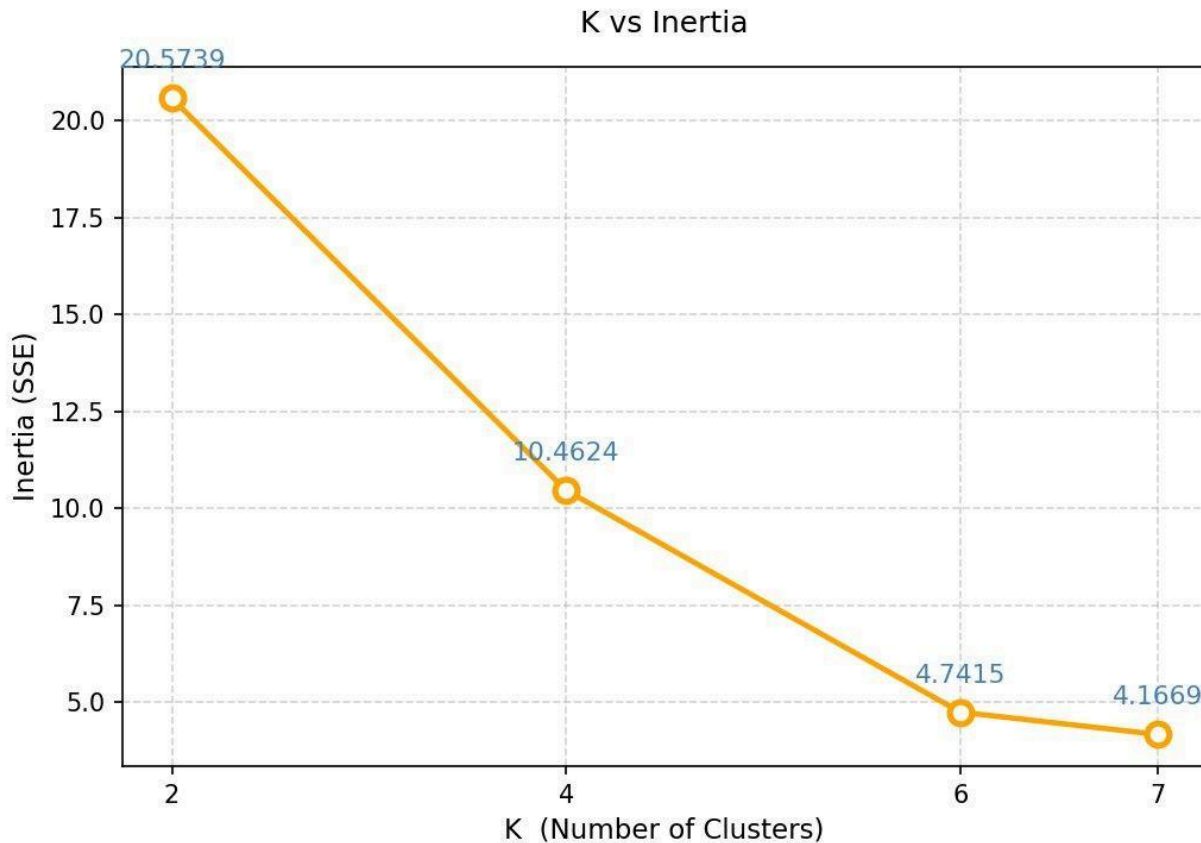


Figure 2: Inertia (SSE) vs. number of clusters K — the bend at K = 6 suggests it as the optimal choice

7. How Inertia Changes with K

- K = 2 → Inertia: 20.57 — Very high. Only two clusters, so many distant points are forced into the wrong group.
- K = 4 → Inertia: 10.46 — Drops by ~10. A big improvement as the horseshoe shape begins to be captured.
- K = 6 → Inertia: 4.74 — Another major drop. Clusters now fit the data shape much more closely.
- K = 7 → Inertia: 4.17 — Very small drop (~0.57). Adding one more cluster gives almost no benefit.

The elbow point is at K = 6, where the rate of decrease becomes much smaller. Going beyond K = 6 provides little improvement while increasing model complexity. This makes K = 6 the most reasonable choice for this dataset.

8. Conclusion

- K-Means was implemented from scratch using only numpy and math — no sklearn or pandas.
- Inertia decreases as K increases, but the rate of decrease slows significantly after K = 6.
- The elbow curve confirms K = 6 as the optimal number of clusters for this dataset.
- K = 2 and K = 4 result in poor separation as they cannot capture the curved structure of the data.

- $K = 7$ over-segments the data with minimal gain in clustering quality.